

DISTRIBUTED SYSTEMS

"A distributed system is a collection of independent computers that appears to its users as a single coherent system. In Spanish, we call it 'Sistema Distribuido.'"

CLUSTERS: "Collection of independent computers"

- Multiple machines (Computers)
- Each can work alone
- Connected by a network

"Appears as a single system"

- Users don't see the complexity
- Like using Google – you don't know which server answers

WHY? KEY CONCEPTS

- **Scalability:** capacity of a system to add more resources without affecting the architecture.
- **Reliability:** Fault Tolerance
- **Latency**
- **Performance**
- **Cluster**
- **Node:** MASTER and SLAVE
- **Load Balancing**
- **Algorithms:** MapReduce
- **Messaging:**
 - Topic
 - Subscription

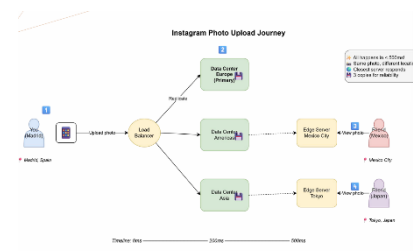
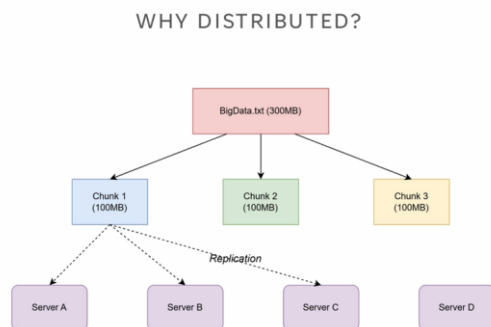
DISTRIBUTED STORAGE SYSTEMS

What is Distributed Storage?

Distributed storage is a method of storing data across multiple physical servers or locations, rather than keeping all data on a single machine. The data appears as one unified system to users, even though it's spread across many devices.

Simple Analogy:

Think of it like storing your belongings in multiple lockers instead of one giant closet. If one locker breaks, you still have access to items in the others.



REPLICATION FACTOR

The replication factor is the number of copies (replicas) of data that a distributed system maintains across different nodes or locations.

It's a fundamental concept in distributed systems that directly impacts:

- Reliability
- Availability
- Performance

CAP THEOREM

The CAP Theorem, also known as Brewer's Theorem (proposed by Eric Brewer in 2000), states that in a distributed data system, it is impossible to simultaneously guarantee all three of the following properties:

- **Consistency:** All nodes see the same data at the same time.
- **Availability:** The system remains operational 100% of the time.
- **Partition Tolerance:** The system continues to operate despite network failures.

The Key Point:

You can only guarantee **2 out of 3** at any given time.

PREGUNTAS Y TF

DISTRIBUTED COMPUTING

Remember: distributed storage

- Data partitioning across nodes
 - Replication for fault tolerance
 - HDFS, distributed databases
 - CAP theorem considerations
-

KEY CONCEPTS (1–5)

1. **Data Locality**
Moving computation to where data resides rather than transferring data across the network.
Principle: *"Moving computation is cheaper than moving data."*
2. **Parallelization**
Breaking tasks into independent subtasks that execute simultaneously across multiple nodes to reduce total execution time.
3. **Fault Tolerance**
System's ability to continue operating despite node or task failures through mechanisms like task re-execution, data replication, and lineage tracking.
4. **MapReduce Paradigm**
Programming model with two phases: Map (transform data independently in parallel) and Reduce (aggregate results by key), abstracting distribution complexity from programmers.
5. **Partitioning**
Dividing data into chunks distributed across nodes using strategies like hash partitioning, range partitioning, or custom partitioning to optimize data locality and performance.
6. **Lazy Evaluation**
Building an execution plan without immediate execution; operations are only triggered by actions, enabling pipeline optimization and minimizing unnecessary data movement.

7. **Shuffle**

Redistributing data across nodes (typically by key) between processing stages. Most expensive operation due to network and disk I/O costs.

8. **In-Memory Computing**

Keeping data in RAM between operations instead of writing to disk, achieving ~100x performance improvement, critical for iterative algorithms.

9. **Lineage**

Tracking the sequence of transformations applied to create a dataset, enabling efficient fault tolerance by recomputing lost data instead of replicating it.

10. **Master-Worker Pattern**

Architectural pattern where a master/coordinator node distributes work and monitors progress while worker/executor nodes perform assigned tasks and report status.

DISTRIBUTED STORAGE vs DISTRIBUTED COMPUTING

Distributed Storage Distributed Computing

WHERE to store data HOW to process data

Data placement Task placement

Replication Parallelization

Focus: Availability Focus: Performance

A task can be seen as a code file or algorithm used to modify the data.

PRINCIPLES

1. Data Locality

- Process data where it's stored
- Minimize network transfer
- *"Moving computation is cheaper than moving data"*

2. Parallelization

- Divide task into subtasks
- Execute simultaneously

- Aggregate results

3. Fault Tolerance

- Nodes will fail
- System must continue
- Re-execute failed tasks

MAP REDUCE PARADIGM

Two main phases:

1. **Map:** Apply function to data chunks independently
2. **Reduce:** Aggregate results from map phase

Origin: Google's solution for distributed computing (2004):

<https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/>

Keys

- Abstraction hides complexity
- Automatic parallelization
- Built-in fault tolerance

MAP REDUCE PARADIGM – FUNCTIONING

Map and Reduce. Image created by the author.

- **Map:**
It takes key/value pair inputs, processes them, and outputs intermediate key/value pairs.
The library will group all values of the same key and pass them to the Reduce tasks.
- **Reduce:**
It receives intermediate values from Map tasks.
The intermediate values are supplied to the Reduce via an iterator.
It then merges the intermediate values from the same key using the logic defined in the Reduce function (e.g., Count, Sum, ...).
The Reduce typically produces at most one output value.
- **Master Node:** Coordinates task assignment
- **Worker Nodes:** Execute map/reduce tasks

MAP REDUCE PARADIGM – Challenges & Solutions

Challenges:

- Which worker processes which data partition?
- What if a worker fails mid-task?
- How to handle stragglers (slow workers)?

Solutions:

- Data locality: Assign tasks to nodes with data
 - Speculative execution: Re-run slow tasks elsewhere
 - Task redundancy: Backup tasks for critical operations
-

MAP REDUCE PARADIGM – Limitations & Timeline

Limitations:

- Disk I/O between stages
- Not suitable for iterative algorithms
- High latency (batch processing)
- Complex multi-stage jobs are cumbersome
- No interactive queries

Timeline:

- 2004: MapReduce paper (Google)
 - 2006: Hadoop MapReduce (Yahoo)
 - 2010: Limitations become clear
 - 2014: Apache Spark gains adoption
 - 2020+: Specialized frameworks
-

BEYOND MR

- **DAG Executions (Directed Acyclic Graph):** Optimization of entire and complex workflows.
- **In Memory Computing:** much faster I/O between stages.

- **Lazy Evaluation**
 - **Partitioning Strategies:** Range, Hash or Custom
-

YARN – YET ANOTHER RESOURCE NEGOTIATOR

YARN Workflow:

1. Client submits application to ResourceManager
2. RM allocates container for ApplicationMaster
3. AM requests containers for tasks
4. RM allocates containers on NodeManagers
5. AM launches tasks in containers
6. Tasks execute and report progress
7. AM reports to RM, client monitors

Architecture:

- **ResourceManager:** Cluster-wide resource allocation
- **NodeManager:** Per-node resource management
- **ApplicationMaster:** Per-application coordination

Separated from processing: **not locked to MapReduce**

Container = CPU + Memory allocation

EDA: EVENT DRIVEN ARCHITECTURES – REAL-TIME DATA SYSTEMS

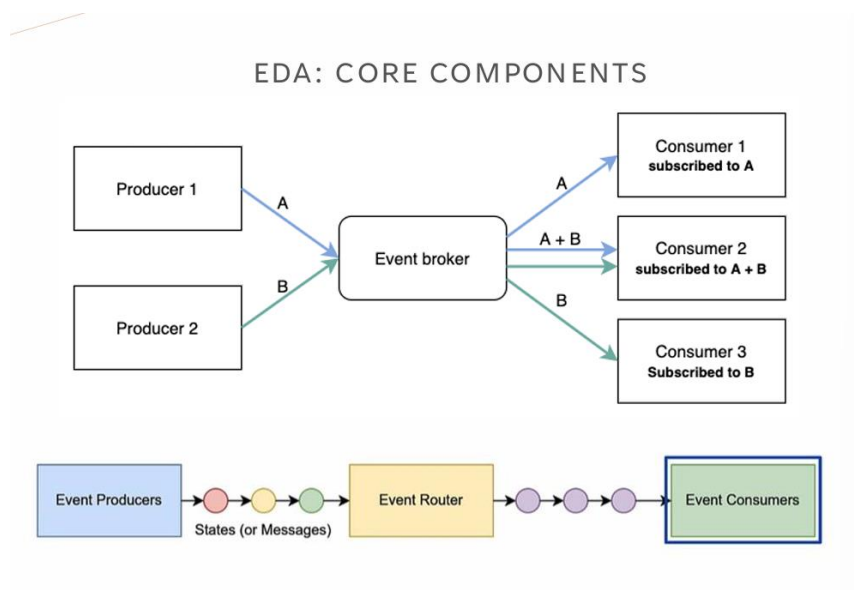
Objectives:

- Understand event-driven architecture
- Compare Kafka, Pub/Sub, and message queues
- Design your own event-based system

- Apply concepts to data engineering scenarios

EDA: KEY CONCEPTS

1. **Event:**
A significant change in state or an occurrence that happens at a specific point in time within a system.
2. **Message:**
A discrete unit of data that is transmitted from one component to another, containing information about an event or a command to be executed.
3. **Producer (Publisher):**
A component or application that creates and sends messages/events to a messaging system.
4. **Consumer (Subscriber):**
A component or application that receives and processes messages/events from a messaging system.
5. **Broker (Message Broker or Router):**
An intermediary program/server that translates, routes, and delivers messages between producers and consumers.
6. **Topic:**
A named logical channel or category to which messages are published and from which consumers can subscribe to receive messages.
7. **Queue:**
A data structure that stores messages in a first-in-first-out (FIFO) order, where each message is typically processed by only one consumer.



EDA: PRODUCERS

CREATES and SENDS the events.

- IoT sensors
- User clicks
- Database changes
- API calls

EDA: BROKERS – THE TRAFFIC CONTROLLERS

The main part of the architecture. Receives the messages and handles delivery.

Sources (examples):

- IoT sensors
- User clicks
- Database changes
- API calls

Broker responsibilities:

- Store messages
 - Route to consumers
 - Ensure delivery
 - Handle failures
-

EDA: CONSUMERS

The main part of the architecture. Receives the messages and handles delivery.

(Same fuentes de eventos en el esquema:)

- IoT sensors
- User clicks
- Database changes
- API calls

Examples of consumers:

- Analytics service
- Database writer
- Notification sender
- ML pipeline

EDA: PUB/SUB PATTERN

- Same message replicates to multiple consumers.
- Subscription pattern.

Topic: abstraction or classification of the messages.

Example: YouTube channel.

- Creator publishes a video → many subscribers are notified.

MESSAGE QUEUE PATTERN

Task Distribution:

- Publisher sends to different classified queues.
- Queues send one by one to all subscriber.
- Example: mailing lists.

HANDS ON: DESIGN EVENT ARCHITECTURE (PUB/SUB)

- Choose an app or service from these at the right

- Find a minimum of six events you want to track
- Design an event driven infrastructure that takes in count the following:
 - Identify Consumers and Publishers.
 - Define at least a TOPIC
 - The events should be consumed by other users, a central app, or both.
 - You must consider both Analytical purposes (a database) and application purposes (other users).

EXAMPLE:

‘Add to Cart’ event, ‘Play’ event, ‘Subscribe channel’ event.

Modelo	PROS	CONS
Event-Driven (Pub/Sub)	Escalable, asíncrono, bajo acoplamiento, fácil añadir servicios	Relación entre servicios compleja, posibles inconsistencias, difícil mantener orden
Request-Response	Simplicidad, consistencia inmediata, fácil depurar	Latencia mayor, acoplamiento alto, fallos parciales, polling caro